

Lab 1 Linux

Note

Before completing this experiment, please refer to "Experiment Environment Setup" to configure the experimental environment.

In this section, we will become familiar with commonly used Linux commands through experimental steps, thereby gaining an initial understanding of the Linux system.

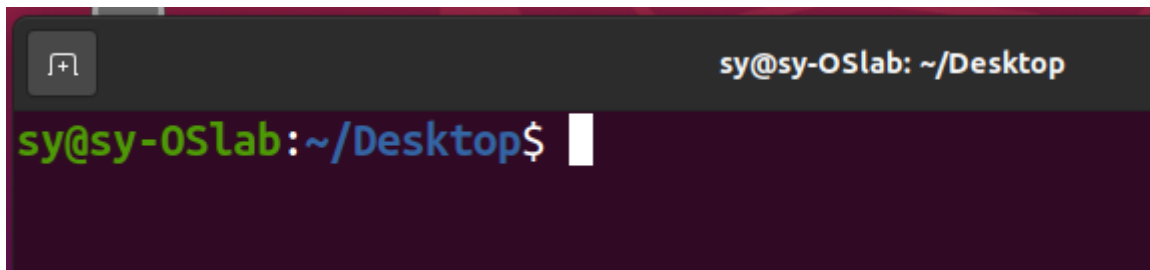
Step 1. Open Terminal

The **Terminal** is a command-line interface that allows users to interact with the operating system by entering text commands. It is one of the core tools in Ubuntu and other Linux-based operating systems, enabling users to perform various tasks such as file management, software installation, system configuration, and more.

Terminal can be opened in the following ways:

- Use the shortcut: `Ctrl + Alt + T`
- Search "Terminal" in "Show Applications"
- Right click on the desktop or folders, choose "Open in Terminal"

The image below shows a basic interface of the Terminal:



- sy-OSlab represents the name of host machine
- sy represents user name
- ~/Desktop represents the current working directory, your commands is executed in this directory
- "~" means Relative path "/home/currentuser", which is the "personal" directory of current user
- \$ means current user is an ordinary user rather than an administrator user
- You can select historical commands through the up and down buttons on the keyboard
- Click tab after entering part of the path or file name to complete it automatically

CUI vs GUI

Cui (command user interface). Users interact with the operating system through text commands, as command line in Windows system and terminal of Linux system. In the experiment of this course, we mainly complete the experiment through CUI.

GUI (graphical user interface). Users interact with the system through various forms of operations on the graphical interface (mouse, keyboard and other input devices), such as computer desktop system and smart mobile phone operation.

Step 2. Learn to use: ls, man, pwd, cd, mkdir, rm, cp, mv, history

ls

list all files(directory) of current directory

```
sy@sy-OSlab:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
s 桌面
```

Please try "ls -l" and "ls -a"

man

For help, you can see the manual of each commands by "man"

try "man ls"

pwd

View the full path of the current directory

```
sy@sy-OSlab:~$ pwd
/home/sy
```

cd

Change working directory

```
sy@sy-OSlab:~$ pwd
/home/sy
sy@sy-OSlab:~$ cd oslab
sy@sy-OSlab:~/oslab$ cd ..
sy@sy-OSlab:~$ pwd
/home/sy
sy@sy-OSlab:~$ cd oslab
sy@sy-OSlab:~/oslab$ cd /
sy@sy-OSlab:/$ pwd
/
sy@sy-OSlab:/$ cd ~
sy@sy-OSlab:~$ pwd
/home/sy
```

"cd ..", change to parent directory

"cd path", change to path

"cd /", change to directory /, "/" is the root directory of Ubuntu

"cd ~", change to the directory of current user(/home/currentuser)

mkdir

New a directory in current directory

```
sy@sy-OSlab:~$ ls
Desktop  Downloads  Pictures  Templates  桌面
Documents Music      Public    Videos
sy@sy-OSlab:~$ mkdir OSlab
sy@sy-OSlab:~$ mkdir oslab
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Pictures  Templates  桌面
Documents Music     oslab  Public    Videos
```

Linux system is strictly case sensitive. The case of the same letter will be regarded as two different letters.

rm

Delete file/diretory

```
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Pictures  Templates  Videos
Documents Music     oslab  Public    test.txt    桌面
sy@sy-OSlab:~$ rm test.txt
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Pictures  Templates  桌面
Documents Music     oslab  Public    Videos
```

When deleting a folder, you may encounter the following errors

```
sy@sy-OSlab:~$ rm oslab
rm: cannot remove 'oslab': Is a directory
```

This is because directory is different from the file. There may be files or directory in the directory, so we need to delete it "recursively". Therefore, when deleting, add the "-r" parameter to delete the directory recursively.

```
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Pictures  Templates  桌面
Documents Music     oslab  Public    Videos
sy@sy-OSlab:~$ rm oslab
rm: cannot remove 'oslab': Is a directory
sy@sy-OSlab:~$ rm -r oslab
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Public  Videos
Documents Music    Pictures  Templates  桌面
```

cp

Copy file/directory

```

sy@sy-OSlab:~/OSlab$ ls
test.txt
sy@sy-OSlab:~/OSlab$ cp test.txt test.txt.bak
sy@sy-OSlab:~/OSlab$ ls
test.txt  test.txt.bak

```

Try copying the folder by the "cp"

mv

Move file/directory to different directory. If **mv** in the same directory, it will rename the file/directory

```

sy@sy-OSlab:~/OSlab$ ls
test.txt  test.txt.bak
sy@sy-OSlab:~/OSlab$ mv test.txt ~
sy@sy-OSlab:~/OSlab$ ls
test.txt.bak
sy@sy-OSlab:~/OSlab$ cd ..
sy@sy-OSlab:~$ la
.bash_history  Documents  .pam_environment  Templates
.bash_logout   Downloads  Pictures           test.txt
.bashrc        .gnupg    .profile          Videos
.cache         .local    Public            .viminfo
.config        Music     .ssh             桌面
Desktop        OSlab    .sudo_as_admin_successful
sy@sy-OSlab:~$ mv test.txt testrename.txt
sy@sy-OSlab:~$ ls
Desktop  Downloads  OSlab  Public  testrename.txt  桌面
Documents Music     Pictures Templates  Videos

```

history

View used commands

Step 3. Learn to use: echo, find, cat, grep, |(pipe), >, >>, <

echo

Print in terminal

```

sy@sy-OSlab:~$ echo $HOME
/home/sy
sy@sy-OSlab:~$ echo "hahaha"
hahaha

```

find

Find files in directory

```

sy@sy-OSlab:~/Desktop$ ls
a.out  hello.c  qemu-5.0.0  qemu-5.0.0.tar.xz  ucireonrv
sy@sy-OSlab:~/Desktop$ find *.c
hello.c

```

You can specify to search by file name, file type, size or other information

cat

View file in terminal

```
sy@sy-OSlab:~/Desktop$ cat hello.c
int main(){
    while(1){
    }
    return 0;
}
```

Related commands: head, tail, more, less

```
sy@sy-OSlab:~/Desktop$ head -n 2 hello.c
int main(){
    while(1){
sy@sy-OSlab:~/Desktop$ tail -n 2 hello.c
    return 0;
}
```

grep

Find qualified strings in the file

```
sy@sy-OSlab:~/Desktop$ cat text
hello world
hello cse
hello os

sy@sy-OSlab:~/Desktop$ grep hello text
hello world
hello cse
hello os
```

| (pipe)

Let the output of the command before | be the input of the command after |

```
sy@sy-OSlab:~/Desktop$ ls
a.out  hello.c  qemu-5.0.0  qemu-5.0.0.tar.xz  text  ucronrv
sy@sy-OSlab:~/Desktop$ ls | grep ll
hello.c
```

> , >> , <

Input / output redirection

```

sy@sy-OSlab:~/Desktop$ ls >test
sy@sy-OSlab:~/Desktop$ ls
a.out  hello.c  qemu-5.0.0  qemu-5.0.0.tar.xz  test  text  ucoreonrv
sy@sy-OSlab:~/Desktop$ cat test
a.out
hello.c
qemu-5.0.0
qemu-5.0.0.tar.xz
test
text
ucoreonrv

```

> can redirect the output to a file. In the figure above, the result of the "ls" is output to the file "test"

Try >>, <

Step 4. Learn to use: sudo, chmod

sudo

Execute commands with administrator privileges

```

sy@sy-OSlab:~/Desktop$ mv test /
mv: cannot move 'test' to '/test': Permission denied
sy@sy-OSlab:~/Desktop$ sudo mv test /
[sudo] password for sy:
sy@sy-OSlab:~/Desktop$ cd /
sy@sy-OSlab:/$ ls
bin      dev      lib      libx32   mnt      root     snap     sys      usr
boot     etc      lib32    lost+found  opt      run      srv      test     var
cdrom    home     lib64    media    proc     sbin     swapfile tmp

```

When we need to execute some instructions, but cannot execute without administrator permission, we can use "sudo"

Don't try "sudo rm -rf /*"

chmod

Modify file or directory permissions

The permission of the file can be viewed through the "ls -l"

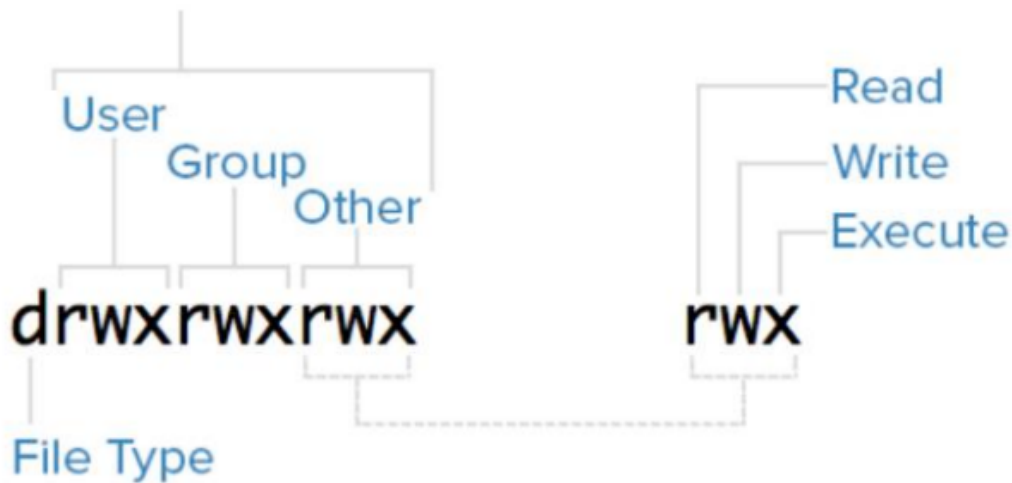
```

sy@sy-OSlab:~/Desktop$ ls -l
total 61008
-rwxrwxr-x  1 sy sy    16464 2月 12 11:30 a.out
-rw-rw-r--  1 sy sy      39 2月 12 11:30 hello.c
drwxr-xr-x 57 sy sy   12288 2月  5 15:09 qemu-5.0.0
-rw-rw-r--  1 sy sy 62426192 4月 29 2020 qemu-5.0.0.tar.xz
-rw-rw-r--  1 sy sy      32 2月 14 11:31 text
drwxrwxr-x  9 sy sy    4096 1月 21 18:31 ucoreonrv

```

drwxrwxr-x represents the permissions of directory ucoreonrv.

Permissions Classes



If you need to modify permissions, you can use "chmod"

```
sy@sy-OSlab:~/Desktop$ ls -l
total 61008
-rwxrwxr-x 1 sy sy 16464 2月 12 11:30 a.out
-rw-rw-r-- 1 sy sy 39 2月 12 11:30 hello.c
drwxr-xr-x 57 sy sy 12288 2月 5 15:09 qemu-5.0.0
-rw-rw-r-- 1 sy sy 62426192 4月 29 2020 qemu-5.0.0.tar.xz
-rw-rw-r-- 1 sy sy 32 2月 14 11:31 text
drwxrwxr-x 9 sy sy 4096 1月 21 18:31 ucronrv
sy@sy-OSlab:~/Desktop$ chmod o+x text
sy@sy-OSlab:~/Desktop$ ls -l |grep text
-rw-rw-r-x 1 sy sy 32 2月 14 11:31 text
sy@sy-OSlab:~/Desktop$ chmod g-w text
sy@sy-OSlab:~/Desktop$ ls -l |grep text
-rw-r--r-x 1 sy sy 32 2月 14 11:31 text
sy@sy-OSlab:~/Desktop$ chmod u=wx text
sy@sy-OSlab:~/Desktop$ ls -l |grep text
--wxr--r-x 1 sy sy 32 2月 14 11:31 text
sy@sy-OSlab:~/Desktop$ chmod 775 text
sy@sy-OSlab:~/Desktop$ ls -l |grep text
-rwxrwxr-x 1 sy sy 32 2月 14 11:31 text
```

As shown above, u\g\o represents user\group\other class user respectively. +,-,= represents add, remove, set permissions.

Chmod 775 can set the permissions of all groups at one time. The numbers 7 and 5 represent binary 111 and 101 respectively, and the numbers on the binary bits represent the corresponding permissions of rwx respectively. For example, 101 represents "1 can r + 0 can not w + 1 can X". Therefore, Chmod 775 represents changing the file permissions to user group readable, writable and executable, group readable, writable and executable, and other readable, writable and executable.

Step 5. installation of vim

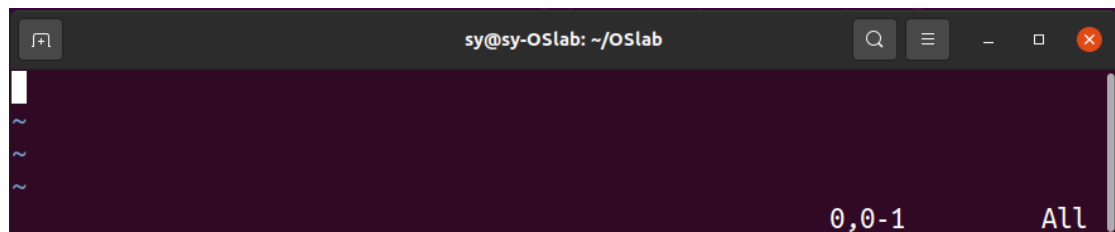
Vim is a highly configurable text editor widely used for programming and text editing. It is an enhanced version of the Vi editor (short for Vi IMproved) and allows users to edit text directly within the terminal.

use "apt-get install vim" to install vim, vim is an editor, which can edit files through terminal

```
sy@sy-OSlab:~/Desktop$ sudo apt-get install vim
[sudo] password for sy:
Reading package lists... Done
Building dependency tree
Reading state information... Done
vim is already the newest version (2:8.1.2269-1ubuntu5.7).
The following packages were automatically installed and are no longer re
quired:
  ibverbs-providers ipxe-qemu libaio1 libcacard0 libfdt1 libibverbs1
  libiscsi7 libpmem1 librados2 librbd1 librdmacm1 libslirp0
  libspice-server1 libusbredirparser1 libvirglrenderer1
  qemu-block-extra qemu-system-common qemu-system-data qemu-system-gui
  qemu-utils seabios
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 87 not upgraded.
```

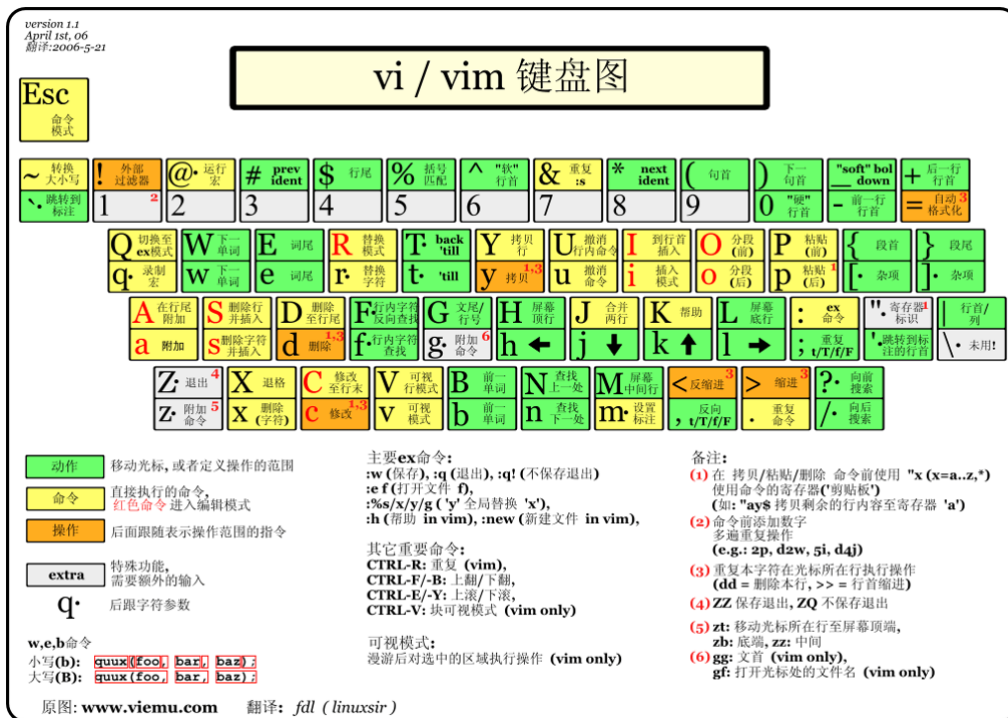
After installation, you can create or open files through "vim"

```
sy@sy-OSlab:~/OSlab$ vim hello.c
```



After opening the file through vim, you will enter the command mode as shown in the figure above. There are three modes in vim:

- Command mode: All user input are command rather than text in this mode
- Insert mode: Click "i" in command mode, you can get into Insert mode. You can edit text in insert mode. Click "Esc" you can come back to command mode.
- Last line mode: Click shift+; , you can get into last line mode, in which mode you can input special commands to the last line, such as "wq" means write and quit.



Step 6. Use gcc to run c programs

vim hello.c

Create and complete a simple dead loop c code through vim

```
sy@sy-OSlab: ~/OSlab
int main(){
    while(1){
    }
    return 0;
}
~
~
-- INSERT --                    5,1    All
```

Save and quit

```
sy@sy-OSlab: ~/OSlab
int main(){
    while(1){
    }
    return 0;
}
~
~
:wq
```

gcc hello.c

Compile the file through gcc and generate an executable file. If the output file name is not specified, the default name of the executable file is a.out. More specific gcc operation process will be practiced in the next experimental class. In this class, we only need to be able to run the simplest c language program.

```
sy@sy-OSlab:~/OSlab$ gcc hello.c
sy@sy-OSlab:~/OSlab$ ls
a.out  hello.c  test.txt.bak
```

./a.out

Run the a.out file through the ". / a.out" instruction, where symbol '.' represents the current working directory.

```
sy@sy-OSlab:~/OSlab$ ./a.out
```

Since we are writing an endless loop program, we can observe that the program enters an endless loop after execution.

Step 7. Learn to use: ps, kill, pstree

ps

Open another terminal and execute the PS instruction to view the list of processes in the current session

```
sy@sy-OSlab:~/Desktop$ ps
  PID TTY          TIME CMD
 5910 pts/1        00:00:00 bash
 5916 pts/1        00:00:00 ps
```

Since the previously run program and the new terminal are not in the same session group, it is not shown in the above figure, but it can be viewed through "ps -a" .

```
sy@sy-OSlab:~/Desktop$ ps -a
  PID TTY          TIME CMD
 1584 tty2        00:00:18 Xorg
 1619 tty2        00:00:00 gnome-session-b
 5899 pts/0        00:15:39 a.out
 5954 pts/1        00:00:00 ps
```

The PID in this list is the process ID. Each process has a different process ID, which is generally allocated in ascending order. However, the number of process numbers is limited and will be recycled.

pstree

View the relationship between processes, and "pstree -p" can display the process tree with process ID

```

├── {gnome-shell-cal}(+)
├── gnome-terminal-(2726)
│   ├── bash(2737)──a.out+
│   ├── bash(5910)──pstree+
│   └── {gnome-terminal}(+)

```

kill

Send an interrupt to the process. The "kill -9 PID" sends a forced termination signal (sigkill), which can be used to kill the process (forced termination)

```
sy@sy-OSlab:~/Desktop$ ps -a
  PID TTY          TIME CMD
 1584 tty2        00:00:21 Xorg
 1619 tty2        00:00:00 gnome-session-b
 5899 pts/0        00:24:31 a.out
 5987 pts/1        00:00:00 ps
sy@sy-OSlab:~/Desktop$ kill -9 5899
sy@sy-OSlab:~/Desktop$ ps -a
  PID TTY          TIME CMD
 1584 tty2        00:00:21 Xorg
 1619 tty2        00:00:00 gnome-session-b
 5988 pts/1        00:00:00 ps
```

Step 8. Learn to use: ctrl+c, ctrl+z, fg

When we run a program and cannot exit, we can also stop the process without sending signals through other terminals.

ctrl+c

Terminate foreground process

```
sy@sy-OSlab:~/OSlab$ ./a.out
^C
```

ctrl+z

Suspend (pause) foreground process

```
sy@sy-OSlab:~/OSlab$ ./a.out
^Z
[1]+  Stopped                  ./a.out
```

```
sy@sy-OSlab:~/OSlab$ ps -l
F S  UID      PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000    2737   2726  0  80   0 -  4913 do_wai pts/0        00:00:00 bash
0 T   1000    6030   2737  1  80   0 -  591 do_sig pts/0        00:00:03 a.out
0 R   1000    6046   2737  0  80   0 -  5013 -      pts/0        00:00:00 ps
```

The suspended foreground process is not killed, but enters the T state. Later in the course, we will learn more about the status of the process.

```
0 R 1000 0040 2737 0 80 0 - 301
sy@sy-OSlab:~/OSlab$ ./a.out
^7
[2]- Stopped ./a.out
sy@sy-OSlab:~/OSlab$
```

The number displayed when a process is suspended is the job number of the process (not the process ID)

fg

You can restore the suspended process to the foreground by using the "fg jobnumber".

```
sy@sy-OSlab:~/OSlab$ fg 2
./a.out
█
```

Note

Next week, you will need to submit your lab reports, so please bring a pen.